PYSPARK COMPLETE NOTES

PHASE 5

# Window Functions & UDFs

Rank, number and accumulate rows without collapsing them, and write your own functions.

YOU WILL

- ✓ Explain the difference between groupBy and a window function
- ✓ Use row_number, rank and dense_rank and predict how ties behave
- ✓ Build running totals and grand totals with rowsBetween
- ✓ Write a UDF, set its return type and know when to avoid one

START HERE IF

Phase 4 (groupBy and aggregation).



Built from a Databricks PySpark tutorial notebook and its course outline.
Clickable contents, links and bookmarks work in any PDF viewer.

# Phase 5 · Window Functions & UDFs

Rank, number and accumulate rows without collapsing them, and write your own functions.

**YOU WILL BE ABLE TO**

- ✓ Explain the difference between groupBy and a window function
- ✓ Use row_number, rank and dense_rank and predict how ties behave
- ✓ Build running totals and grand totals with rowsBetween
- ✓ Write a UDF, set its return type and know when to avoid one

**START HERE IF**

Phase 4 (groupBy and aggregation).

**IN THIS PDF**

Explanations, diagrams and code from the notebook · sections tagged **Beyond the notebook** · a decoder for every term · interview questions · a data-engineering lens · a self-test with answers · a glossary

## How to read the boxes

| | |
|---|---|
| **IN PLAIN WORDS**<br>A jargon-free explanation of the idea. | **DECODE THE CODE**<br>Meaning of every function, option and symbol in the code above (first time it appears). |
| **TIP**<br>A habit that makes your code cleaner, faster or safer. | **WATCH OUT**<br>A trap that gives a wrong or surprising result. |
| **NOTEBOOK OBSERVATION**<br>Something specific noticed in the course notebook. | **DATA ENGINEERING LENS**<br>How the skill is used in real pipelines (summary page at the end). |
| **EXTRA**<br>Extra knowledge that is not in the notebook. | **BEYOND THE NOTEBOOK**<br>A tag under a heading: new material added to the notebook's topics, ending with an Interview corner. |

CONTENTS (click an entry to jump to it)

5.1 What is a window function? . . . 3
5.2 row_number() . . . 4
5.3 rank() vs dense_rank() . . . 4
5.4 Cumulative sum and window frames . . . 5
5.5 User defined functions (UDF) . . . 7
From notebook to pipeline . . . 9
Glossary . . . 9
Check yourself . . . 11
Answers and solutions . . . 12

## 5.1 What is a window function?

**IN PLAIN WORDS**

A **groupBy** collapses many rows into one per group. A **window function** looks at a set of related rows (the window) and attaches a result to **every row**, without collapsing anything. It answers questions like "what number is this row in the order?" or "what is the running total up to this row?"

PYSPARK

```
from pyspark.sql.window import Window
```

DECODE THE CODE

| | |
|---|---|
| `from pyspark.sql.window import Window` | Imports the Window class used to define windows. |
| `Window` | Defines the group of rows a window function looks at. |

The shape of every window expression is `function().over(Window.<definition>)`, where the window definition can combine:

**groupBy - collapses rows**

| grp | val |
|---|---|
| A | 10 |
| A | 20 |
| B | 5 |
| B | 15 |

→

| grp | sum |
|---|---|
| A | 30 |
| B | 20 |

4 rows in -> 2 rows out

*the individual rows are gone*

**window function - keeps every row**

| grp | val |
|---|---|
| A | 10 |
| A | 20 |
| B | 5 |
| B | 15 |

→

| grp | val | n |
|---|---|---|
| A | 10 | 1 |
| A | 20 | 2 |
| B | 5 | 3 |
| B | 15 | 4 |

4 rows in -> 4 rows out

*plus a new column, e.g. row_number()*

| Part | Meaning | Used in the notebook? |
|---|---|---|
| `Window.orderBy(...)` | Sort order inside the window. Needed for ranking and running totals. | Yes, always |
| `Window.partitionBy(...)` | Split the data into independent groups first (like a groupBy), and compute inside each group. | **No** - see the warning below |
| `.rowsBetween(start, end)` | The **frame**: exactly which rows around the current row are included. | Yes, in cumulative sums |

**NO PARTITIONBY MEANS ONE GIANT WINDOW**

All the notebook's windows sort the **whole** DataFrame as a single group. Spark warns that all data is moved to a single partition, which is slow on large data. That is fine for learning on a small file; in real work add `partitionBy`, for example `Window.partitionBy('Outlet_Type').orderBy('Item_MRP')`.

## 5.2 row_number()

PYSPARK

```
df.display()
```

DECODE THE CODE

| | |
|---|---|
| `.display()` | Databricks' table viewer. It is an **action**, so it makes Spark run the job. |

PYSPARK

```
df.withColumn('rowCol',row_number().over(Window.orderBy('Item_Identifier'))).display()
```

DECODE THE CODE

| | |
|---|---|
| `.withColumn(name, expression)` | **Adds** a column, or **replaces** the one with the same name. |
| `row_number()` | Numbers rows 1, 2, 3 ... in window order. Always unique. |
| `.over(window)` | Applies the function across the window instead of collapsing rows. |
| `.orderBy(col)` | Sort order **inside** the window - needed for numbering and running totals. |

`row_number()` numbers the rows 1, 2, 3 ... in the window's order. Every row gets a **unique** number, even when the sort column has ties - for tied rows the one that gets the smaller number is arbitrary. The new column is called `rowCol`.

> CLASSIC USE: PICK THE FIRST ROW OF EACH GROUP
> Number the rows with partitionBy + orderBy, then keep `rowCol == 1`. That gives "the top item per outlet" and similar.

## 5.3 rank() vs dense_rank()

PYSPARK · LINE BREAK ADDED FOR THE PAGE, CODE UNCHANGED

```
df.withColumn('rank',rank().over(Window.orderBy(col('Item_Identifier').desc())))\
        .withColumn('denseRank',
                    dense_rank().over(Window.orderBy(col('Item_Identifier').desc()))).display()
```

DECODE THE CODE

| | |
|---|---|
| `rank()` | Rank with ties sharing a number; the next number **skips** (1, 2, 2, 4). |
| `col('name')` | Refers to a column by name so you can do something with it (rename, compare, calculate). |
| `.desc() / .asc()` | Sort direction for one column: descending / ascending. |
| `\ (backslash)` | Line continuation: the statement carries on in the next line. Nothing may follow it on the same line. |
| `dense_rank()` | Rank with ties sharing a number and **no gaps** (1, 2, 2, 3). |

The same item id appears on many rows (the same product sold at several outlets), so there are plenty of **ties**. How ties are numbered is the difference between the three functions:

ORDERED BY ITEM_IDENTIFIER DESCENDING – VALUES TIE

| Item_Identifier | row_number() | rank() | dense_rank() |
|---|---|---|---|
| C | 1 | 1 | 1 |
| B | 2 | 2 | 2 |
| B | 3 | 2 | 2 |
| A | 4 | **4** | **3** |

row_number: always unique,
ties still get different numbers

rank: ties share a number,
then it SKIPS (1, 2, 2, 4)

dense_rank: ties share a number,
NO gaps (1, 2, 2, 3)

*Olympic medals: two silvers -> no bronze (rank). Position by distinct value (dense_rank).*

SMALL ILLUSTRATION OF TIE HANDLING

| Item_Identifier (desc) | row_number | rank | dense_rank |
|---|---|---|---|
| C | 1 | 1 | 1 |
| B | 2 | 2 | 2 |
| B | 3 | 2 | 2 |
| A | 4 | 4 | 3 |

| Function | Tied rows | Numbers after a tie | Think of it as |
|---|---|---|---|
| `row_number()` | Different numbers | Continue 1, 2, 3, 4 … | A unique ID per row |
| `rank()` | **Same** number | **Skip** ahead: 1, 2, 2, **4** | Olympic medals - two silvers, no bronze |
| `dense_rank()` | **Same** number | **No gaps**: 1, 2, 2, **3** | Position by distinct value |

## 5.4 Cumulative sum and window frames

### A first running total

PYSPARK · LINE BREAK ADDED FOR THE PAGE, CODE UNCHANGED

```
df.withColumn('dum',sum('Item_MRP').over(Window.orderBy('Item_Identifier')
              .rowsBetween(Window.unboundedPreceding,Window.currentRow))).display()
```

DECODE THE CODE

| | |
|---|---|
| `sum(col)` | Total of a column (Spark's own *sum*, available through the import *). |
| `.rowsBetween(start, end)` | The **frame**: exactly which rows around the current row are included. |
| `Window.unboundedPreceding / currentRow / unboundedFollowing` | Frame boundaries: from the first row, the current row, through the last row. |

For each row this adds up `Item_MRP` from the very first row down to the current row, in Item_Identifier order. A **frame** is that range of rows. Its ends are named positions:

| Name | Means |
|---|---|
| `Window.unboundedPreceding` | from the first row of the window |
| `Window.currentRow` | up to the row being calculated |
| `Window.unboundedFollowing` | through the last row of the window |

The notebook then compares three ways of writing a cumulative sum ordered by `Item_Type`:

### A - no frame given

PYSPARK

```
df.withColumn('cumsum',sum('Item_MRP').over(Window.orderBy('Item_Type'))).display()
```

### B - frame: first row up to the current row

PYSPARK · LINE BREAK ADDED FOR THE PAGE, CODE UNCHANGED

```
df.withColumn('cumsum',sum('Item_MRP').over(Window.orderBy('Item_Type')
              .rowsBetween(Window.unboundedPreceding,Window.currentRow))).display()
```

## C - frame: the whole window

PYSPARK · LINE BREAK ADDED FOR THE PAGE, CODE UNCHANGED

```
df.withColumn('totalsum',sum('Item_MRP').over(Window.orderBy('Item_Type')
              .rowsBetween(Window.unboundedPreceding,Window.unboundedFollowing))).display()
```

Imagine three rows ordered by Item_Type with these prices. The two Dairy rows are **ties** on the sort column:

ILLUSTRATION WITH MADE-UP PRICES

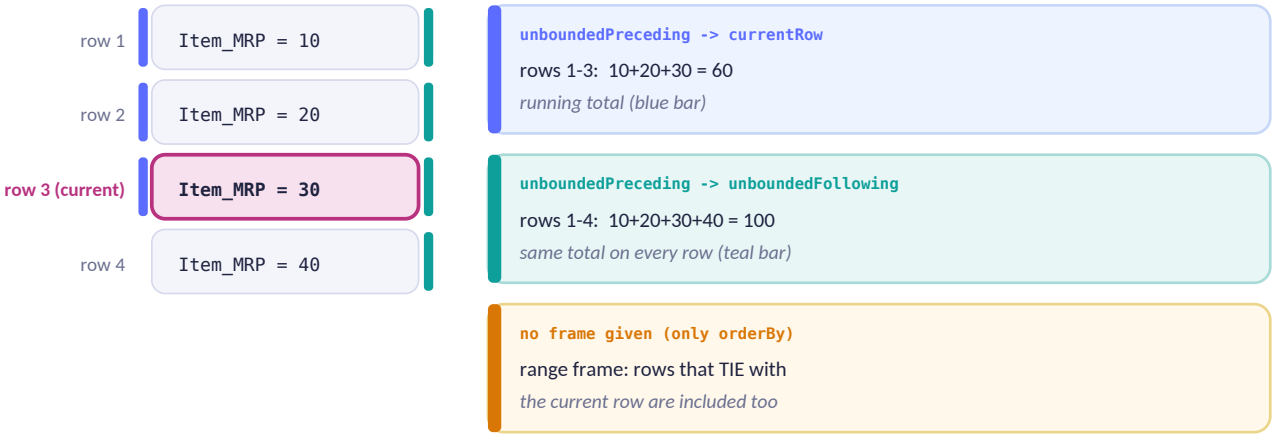
| Item_Type | Item_MRP | A: no frame | B: rowsBetween(start, current) | C: start to end |
|---|---|---|---|---|
| Dairy | 10 | 30 | 10 | 60 |
| Dairy | 20 | 30 | 30 | 60 |
| Meat | 30 | 60 | 60 | 60 |

- **A** (no frame): when an order is given but no frame, Spark uses a **range** frame, which includes all rows that tie with the current row. Both Dairy rows therefore show 30.
- **B** adds row by row (a **rows** frame), so tied rows differ: 10 then 30. Which Dairy row comes first is arbitrary, so the 10 and 30 could swap.
- **C** covers the whole window, so every row shows the same grand total - useful for percentages of the total.

**USE ROWSBETWEEN FOR A TRUE RUNNING TOTAL**

With ties in the order column, the default frame (A) gives equal values to tied rows and looks like it "jumped". If you want one step per row, state the frame explicitly as in B.

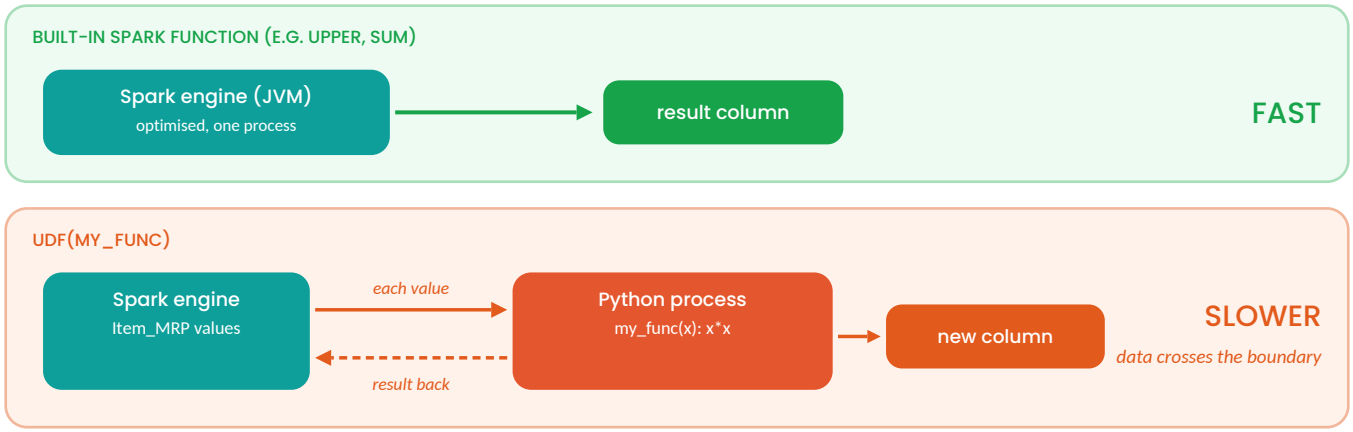
4 ROWS ORDERED BY ITEM_TYPE; WE CALCULATE ROW 3 (THE CURRENT ROW)




*A frame says exactly which rows around the current row are added up.*

# 5.5 User defined functions (UDF)

When no built-in Spark function does what you need, you can write the logic as an ordinary **Python function** and register it so Spark can apply it to every row of a column. That is a **UDF**.




## Step 1 - write a Python function

PYSPARK

```
def my_func(x):
    return x*x
```

DECODE THE CODE

| | |
|---|---|
| `def name(args):` | Defines an ordinary Python function. |

## Step 2 - register it as a UDF and use it

PYSPARK

```
my_udf = udf(my_func)
```

DECODE THE CODE

| | |
|---|---|
| `udf(my_func)` | Wraps a Python function so Spark can apply it to a column. Returns **string** unless you give a type. |

PYSPARK

```
df.withColumn('mynewcol',my_udf('Item_MRP')).display()
```

WHAT THE NEW COLUMN HOLDS (SQUARES OF Item_MRP, ROUNDED HERE)

| Item_MRP | mynewcol |
|---|---|
| 249.8092 | 62404.6... |
| 48.2692 | 2329.9... |
| 141.618 | 20055.6... |

- `udf(my_func)` wraps the Python function so it can be called on a column, like a built-in Spark function.
- Spark sends each value of `Item_MRP` to `my_func`, and the returned value fills the new column.

> **A UDF RETURNS TEXT UNLESS YOU SAY OTHERWISE**
> The notebook's `udf(my_func)` has no return type, so Spark assumes **StringType**. `mynewcol` holds numbers stored as text. Declare the type to get a real number column:

PYSPARK · EXTRA

```
my_udf = udf(my_func, DoubleType())
```

DECODE THE CODE

| | |
|---|---|
| `StringType() / IntegerType() / DoubleType()` | Type objects for text, whole numbers and decimal numbers. |

### PERFORMANCE AND NULLS

UDFs are **slow**: every value travels from Spark's engine to a Python process and back, and Spark cannot optimise inside your function. Prefer built-in functions whenever one exists; save UDFs for logic that truly has no built-in version.

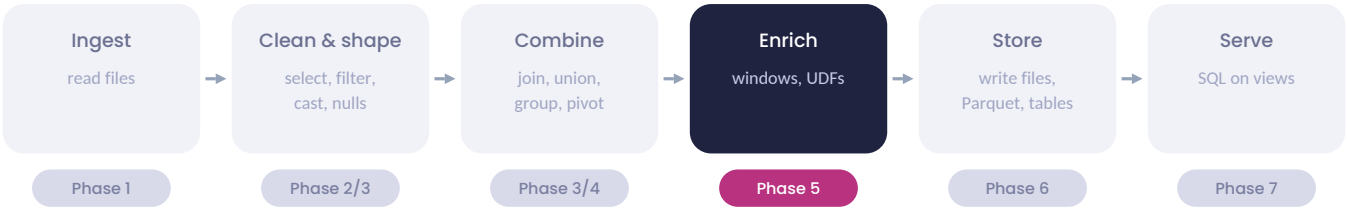
A null value arrives in Python as `None`, and `None * None` raises an error that would fail the whole job. A safe version: `return x*x if x is not None else None`.

### PHASE 5 CHECKPOINT

- I can explain the difference between groupBy (collapses rows) and a window function (keeps every row).
- I can write `function().over(Window.orderBy(...))` and I know what partitionBy would add.
- I can tell row_number, rank and dense_rank apart on tied values.
- I can build running totals and grand totals with rowsBetween, and I understand why ties change the default result.
- I can write a UDF in two steps, set its return type and I know when not to use one.

# From notebook to pipeline

Data engineers use these same skills inside **pipelines**. Here is where this phase sits, and how it is used in practice.




DATA ENGINEERING LENS

A common pipeline pattern: number rows with **row_number** over a window, then keep rank 1 - for example the top row per group.

Running totals (rowsBetween) and grand totals are typical reporting columns that keep every detail row.

On real data add **partitionBy**; a window with only orderBy moves all rows to a single partition.

Treat UDFs as a **last resort**: built-in functions are optimised by Spark and are faster.

# Glossary

**Concepts** from this phase, then every **code term** the decoder explained, in the order you met them.

| Concept | In plain words |
|---|---|
| **Window function** | A calculation over related rows that keeps every row. |
| **Window specification** | orderBy, optionally partitionBy and a frame. |
| **Frame** | The exact rows around the current row included in the calculation (rowsBetween). |
| **Tie** | Rows with equal values in the ordering column. |
| **UDF** | User defined function: your own Python function applied to a column. |

## Code terms decoded in this phase

| Code term | Meaning |
|---|---|
| `from pyspark.sql.window import Window` | Imports the Window class used to define windows. |
| `Window` | Defines the group of rows a window function looks at. |
| `.display()` | Databricks' table viewer. It is an **action**, so it makes Spark run the job. |
| `.withColumn(name, expression)` | **Adds** a column, or **replaces** the one with the same name. |
| `row_number()` | Numbers rows 1, 2, 3 … in window order. Always unique. |
| `.over(window)` | Applies the function across the window instead of collapsing rows. |
| `.orderBy(col)` | Sort order **inside** the window - needed for numbering and running totals. |
| `rank()` | Rank with ties sharing a number; the next number **skips** (1, 2, 2, 4). |
| `col('name')` | Refers to a column by name so you can do something with it (rename, compare, calculate). |
| `.desc() / .asc()` | Sort direction for one column: descending / ascending. |
| `\ (backslash)` | Line continuation: the statement carries on in the next line. Nothing may follow it on the same line. |
| `dense_rank()` | Rank with ties sharing a number and **no gaps** (1, 2, 2, 3). |
| `sum(col)` | Total of a column (Spark's own *sum*, available through the import *). |
| `.rowsBetween(start, end)` | The **frame**: exactly which rows around the current row are included. |
| `Window.unboundedPreceding / currentRow / unboundedFollowing` | Frame boundaries: from the first row, the current row, through the last row. |
| `def name(args):` | Defines an ordinary Python function. |
| `udf(my_func)` | Wraps a Python function so Spark can apply it to a column. Returns **string** unless you give a type. |
| `StringType() / IntegerType() / DoubleType()` | Type objects for text, whole numbers and decimal numbers. |

# Check yourself

Try each question first, then click **Show answer**. Every answer has a **back** link that returns you to the question.

## Questions

Q1 What is the difference between `rank()` and `dense_rank()`? Show answer

Q2 What happens when a window has `orderBy` but no `partitionBy`? Show answer

Q3 What does `rowsBetween(Window.unboundedPreceding, Window.currentRow)` mean? Show answer

Q4 Why can a cumulative sum without a frame show the same value for several rows? Show answer

Q5 Why are UDFs slow, and what type does `udf(my_func)` return by default? Show answer

## Hands-on tasks

Task 1 For each Outlet_Type, number the rows by Item_MRP (highest first) and keep only row 1. Show solution

Task 2 Create a UDF that squares Item_MRP and returns a real number (not text). Show solution

# Answers and solutions

A1 Both give tied rows the same number. rank then **skips** numbers (1, 2, 2, 4); dense_rank does not (1, 2, 2, 3). back to question

A2 All rows form one window and are moved to a single partition - fine for learning, slow on big data. back to question

A3 The frame runs from the first row of the window up to the current row - a running total when used with sum. back to question

A4 With only an orderBy Spark uses a range frame, which includes all rows that **tie** with the current row. back to question

A5 Every value is sent from Spark to a Python process and back, and Spark cannot optimise inside the function. The default return type is **StringType**. back to question

## Task solutions

Solution 1 back to task

SOLUTION

```
w = Window.partitionBy('Outlet_Type').orderBy(col('Item_MRP').desc())

df.withColumn('rowCol', row_number().over(w)).filter(col('rowCol') == 1).display()
```

Solution 2 back to task

SOLUTION

```
def my_func(x):
    return x*x if x is not None else None

my_udf = udf(my_func, DoubleType())

df.withColumn('mynewcol', my_udf('Item_MRP')).display()
```

PREVIOUS
← Phase 4 · Advanced Transformations

NEXT
Phase 6 · Writing Data & Delta Lake →

Tip: keep all nine PDFs in the same folder so the previous / next links open the right file.